

Project QuantumDefense: Enterprise DevOps Architecture & Engineering Manual

Integrated Multi-Domain Command & Control (C2) Platform

Saurabh Yadav

Roll Number: 150096724010

AWS / EKS / Terraform / Jenkins / Vault / Prometheus / Grafana / ELK / React

Repository: <https://github.com/HakashiKatake/QuantumDefence>

■ Problem Statement

Traditional military command centers suffer from:

- 1. **Fragmented Legacy Infrastructure:** Disparate systems with incompatible protocols
- 2. **Information Silos:** Delayed threat identification due to isolated data sources
- 3. **Manual Processes:** Time-consuming certificate rotation, manual deployments
- 4. **Security Vulnerabilities:** Hardcoded credentials, plaintext secrets in configs
- 5. **Operational Blind Spots:** Lack of real-time telemetry from deployed assets
- 6. **Cost Inefficiencies:** Multiple redundant database instances, manual scaling
- 7. **No Disaster Recovery:** Single points of failure with extended recovery times

■ Proposed Solution & How It Was Solved

The QuantumDefense platform delivers:

---	---
Unified Dashboard	React 19 + Leaflet.js map aggregating all domain data
Real-time Telemetry	WebSocket (Socket.IO) pushing updates every 3 seconds
Automated Threat Assessment	Classification engine with severity scoring (<500ms)
Zero-Trust Security	HashiCorp Vault dynamic secret retrieval at runtime
CI/CD Automation	Jenkins pipeline with auto-build, test, deploy
Infrastructure as Code	Terraform modules for reproducible AWS deployments
Observability Stack	ELK for logs, Prometheus/Grafana for metrics
Disaster Recovery	Cross-region failover with RTO < 15 min, RPO < 5 min

■■ Technology Stack

---	---	---	---
-----	-----	-----	-----

Frontend	React	19	UI framework
Build	Vite	6	Fast bundler
Styling	Tailwind CSS	4	Utility-first CSS
Backend	Node.js	22 LTS	Runtime
Framework	Express	4.x	HTTP server
Database	PostgreSQL	18 / 16	Relational store
ORM	Prisma	6.x	Schema management
Auth	JWT	-	Token auth
Real-time	Socket.IO	4.x	WebSockets
Container	Docker	-	Containerization
Orchestration	Kubernetes	1.36	Container mgmt
Cloud	AWS	-	EKS, RDS, ECR
Secrets	HashiCorp Vault	-	Secret management
CI/CD	Jenkins	LTS	Pipeline automation
Monitoring	Prometheus + Grafana	latest	Metrics & dashboards
Logging	ELK Stack	8.x	Centralized logs
Infrastructure	Terraform	1.x	IaC

■ Module Description

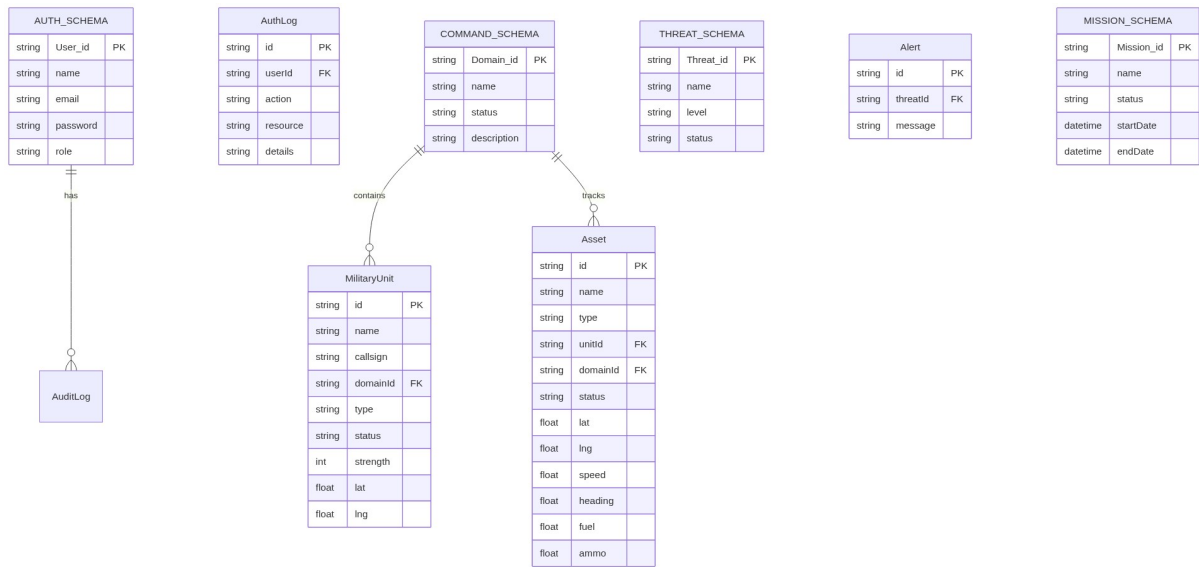
---	---	---	---
auth-service	services/auth-service/	User registration, login, JWT issuance, audit logging	4001
command-service	services/command-service/	Domain management, asset telemetry, unit tracking, WebSocket server	4002

threat-service	services/threat-service/ 	Threat detection, alert generation, WebSocket server	4003
mission-service	services/mission-service/ 	Mission directives, state machine, scheduling	4004
frontend	frontend/ 	React SPA with Leaflet map, dashboards, forms	3000/80
gateway	gateway/ 	Nginx reverse proxy, WebSocket upgrades, SSL termination	80
vault	vault/ 	Secret storage, dynamic credential retrieval, policy enforcement	8200

■ Project Directory Structure

```
QuantumDefence/  
|-- services/  
|   |-- auth-service/      # Node.js + Express (Port 4001) - JWT authentication & audits  
|   |-- command-service/   # Node.js + WebSockets (Port 4002) - Telemetry & asset simulation  
|   |-- threat-service/    # Node.js + WebSockets (Port 4003) - Detection & live alerts  
|   |-- mission-service/   # Node.js + Express (Port 4004) - Directives state machine  
|-- frontend/              # React 19 + Vite 6 Single-Page Application (Tailwind CSS v4)  
|-- gateway/               # Nginx API Gateway routing and WebSocket proxies  
|-- docker/                # Local Docker Compose multi-container templates  
|-- terraform/             # Infrastructure provisioning modules (AWS VPC, EKS, RDS, ECR)  
|-- kubernetes/            # Production manifests (Deployments, Ingress, HPA, ConfigMaps)  
|-- monitoring/            # Prometheus scrapers, alert rules, and Grafana datasources  
|-- logging/               # ELK Logstash pipelines and configurations  
|-- vault/                 # HashiCorp Vault access policies and init scripts  
|-- docs/                  # Extensive project documentation & screenshots  
|-- scripts/               # Provisioning, seeding, and cleanup scripts
```

■■ Database Design



Schema Details

---	---	---
auth	User	id, name, email, password, role, createdAt, updatedAt
auth	Session	id, userId, token, expiresAt
auth	AuditLog	id, userId, action, resource, details, createdAt
command	Domain	id, name, status, description
command	MilitaryUnit	id, name, callsign, domainId, type, status, strength, lat, lng
command	Asset	id, name, type, unitId, domainId, status, lat, lng, speed, heading, fuel, ammo
threat	Threat	id, name, level, status, description
threat	Alert	id, threatId, message, severity, createdAt
mission	Mission	id, name, status, startDate, endDate, description

■ Architectural Trade-offs & Engineering Decisions

I made several critical architectural trade-offs during the implementation of the platform:

1. Shared PostgreSQL Instance with Logical Schema Isolation

- **Trade-off:** Standard microservice design dictates a "database-per-service" pattern. However, hosting four separate database instances in Amazon RDS (and locally in Docker Compose) creates significant resource overhead and increases monthly AWS costs.
- **Decision:** I deployed a single PostgreSQL database instance but enforced strict logical isolation. Each service connects via a database URL referencing a distinct schema (`?schema=auth`, etc.). No table joins or cross-database transactions are permitted. Cross-service data dependencies are resolved entirely at the API tier.

2. WebSockets (Socket.IO) vs. Short Polling for Telemetry

- **Trade-off:** Tactical maps require high-frequency updates. Standard HTTP short polling creates excessive network traffic, burns database read capacity, and introduces lag.
- **Decision:** I implemented persistent WebSockets via Socket.IO. The Command Service broadcasts simulated asset coordinate changes every 3 seconds to all connected clients over a single persistent TCP socket. This minimizes network overhead and achieves near-instantaneous tactical updates.

3. Client-Side JWT Validation vs. Gateway Token Introspection

- **Trade-off:** When a request hits the API Gateway, validating the JWT at the gateway reduces downstream load but centralizes the verification logic. Conversely, letting microservices validate tokens themselves increases code duplication but allows for finer-grained, service-level RBAC.
- **Decision:** Microservices fetch JWT signing keys dynamically from HashiCorp Vault on startup and perform stateless verification. A lightweight `/api/auth/verify` endpoint is provided on the Auth Service for service-to-service validation when secondary authorization is required.

■ Core Technical Challenges & Resolutions

During the implementation and testing phases, I encountered and resolved several complex bugs:

1. PostgreSQL 18 Mount & PGDATA Directory Collisions

- **Problem:** In PostgreSQL 18, mounting the host volume directly to `/var/lib/postgresql/data` caused container initialization to fail because Postgres expects database cluster files to live inside a major-version-specific folder layout (`/var/lib/postgresql/data/pgdata`).
- **Resolution:** I modified the Docker mount point to target `/var/lib/postgresql` and set the environment variable `PGDATA=/var/lib/postgresql/data/pgdata`. This allowed Postgres to safely initialize its sub-directory layout.

2. Prisma Database Push Schema Collisions

- **Problem:** Because Prisma manages the database schema declaratively, running `npx prisma db push` from one service dropped the tables owned by the other services. Prisma attempted to make the target database match the single service's schema.
- **Resolution:** I isolated each service into a dedicated database schema. The database connection strings were updated to append `?schema=`. I adjusted the migration and seed scripts to run under explicit schema namespaces, preventing cross-service schema corruption.

3. Docker Compilation Failures due to Lockfile Desync

- **Problem:** Adding the `jsonwebtoken` dependency to the microservices' source code without running `npm install` on the host created a discrepancy between `package.json` and `package-lock.json`. When Docker ran `npm ci` during the multi-stage build, it aborted due to lockfile desync.
- **Resolution:** I executed localized dependency updates in each service folder, synchronized the lockfiles, and updated the Docker build contexts to explicitly copy the verified lockfiles prior to running the clean install step.

4. API Gateway Welcome Page Routing Override

- **Problem:** Accessing `http://localhost:80` returned the default Nginx welcome page instead of serving the React application. Nginx prioritizes its default server block over custom location mappings if custom static paths are misconfigured.
- **Resolution:** I removed the default Nginx virtual host configurations from the container, updated my Custom `nginx.conf` to host the build output mounted at `/usr/share/nginx/html`, and enabled fallbacks for React Router Single-Page Application routing using `try_files $uri $uri/ /index.html`.

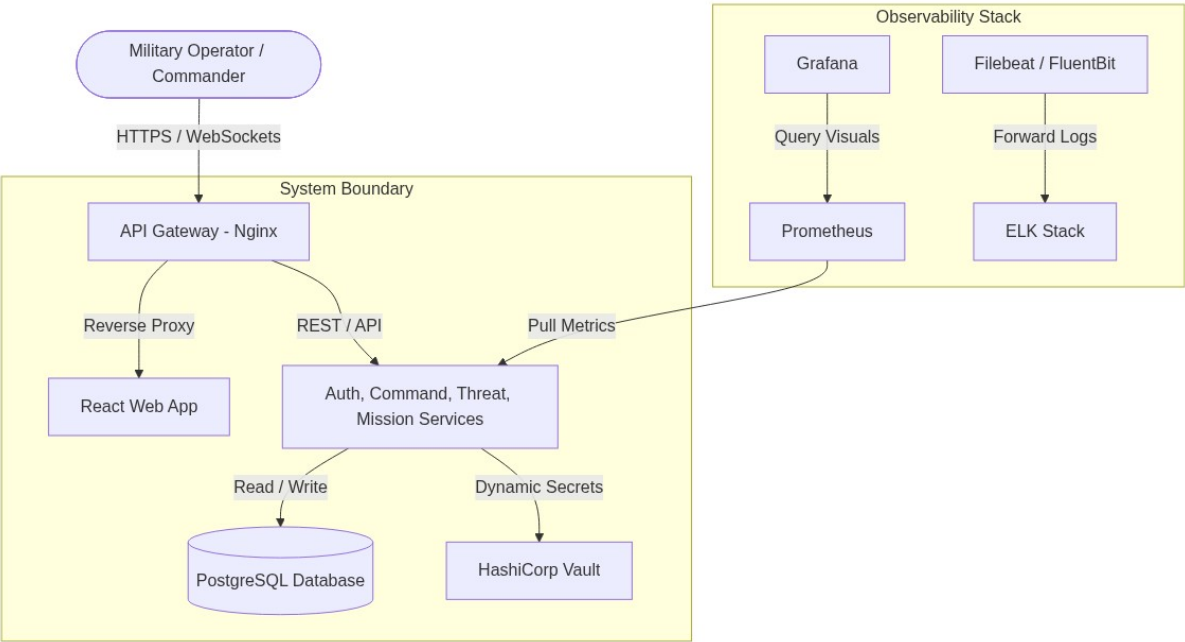
5. Failed ECR Repository Teardown during Cleanup

- **Problem:** When running `prod-cleanup.sh`, the terraform destroy step failed because ECR repositories cannot be deleted if they contain images. This left orphaned AWS resources that continued to accrue charges.
- **Resolution:** I added `force_delete = true` to the `aws_ecr_repository` resource blocks in `terraform/main.tf` and `force_destroy = true` to the S3 buckets. This ensures that Terraform automatically purges repository images and bucket objects before destroying the resources.

■ System Design & Diagrams

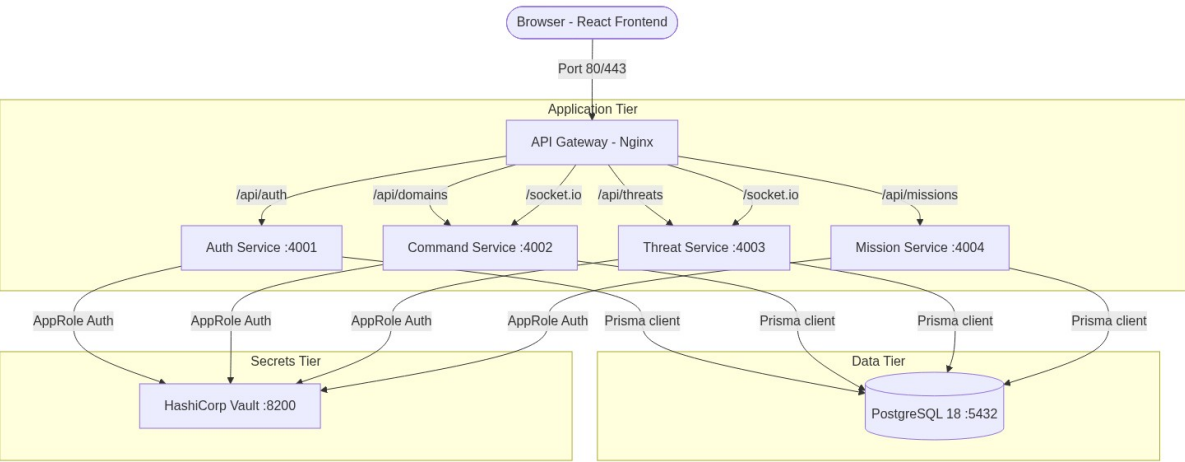
1. System Context Diagram (C4 Context)

The diagram below shows the high-level boundary of the QuantumDefense system:



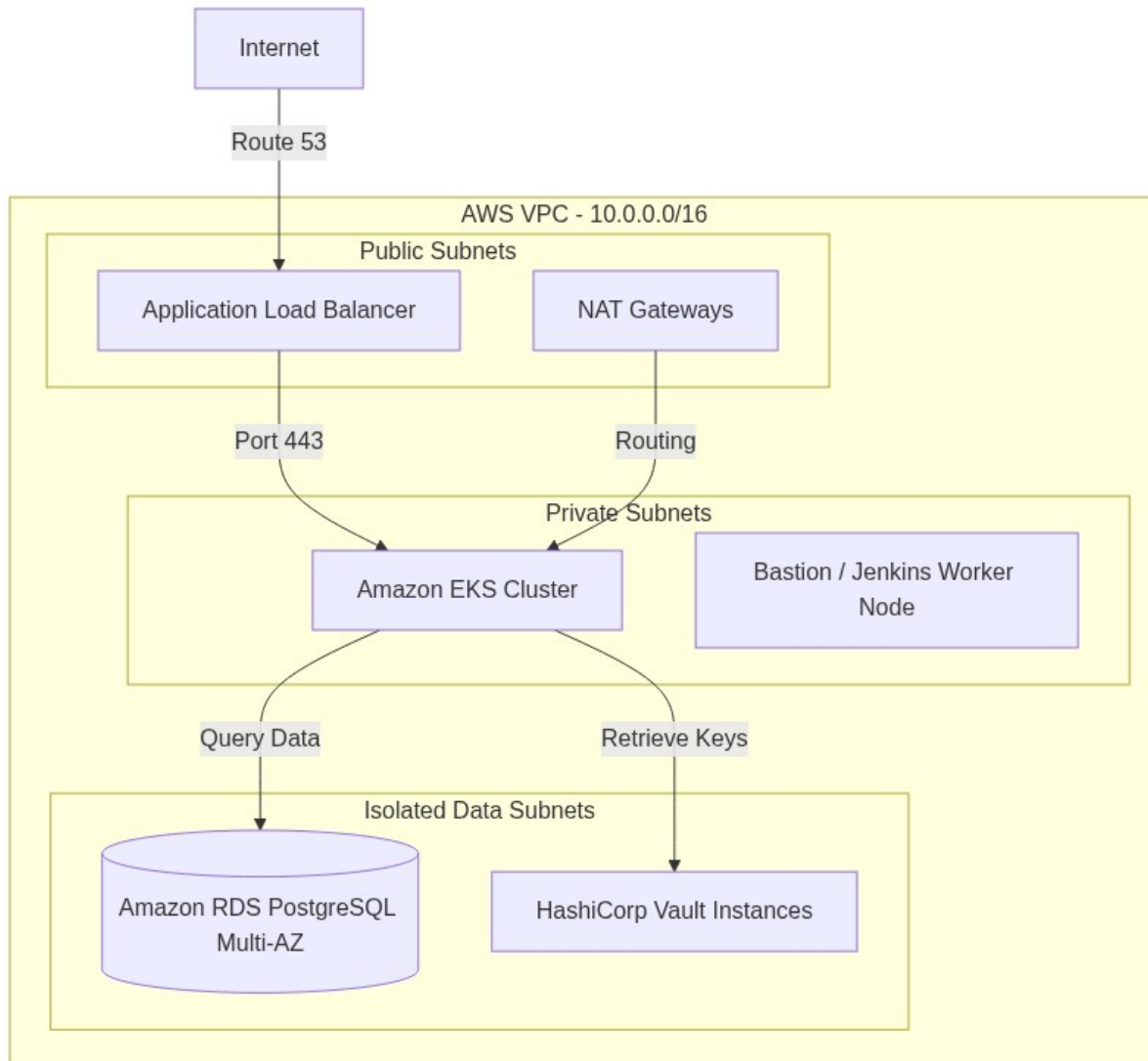
2. Container Architecture (C4 Container Diagram)

This diagram details the interfaces, port bindings, and communication pathways of the container components:



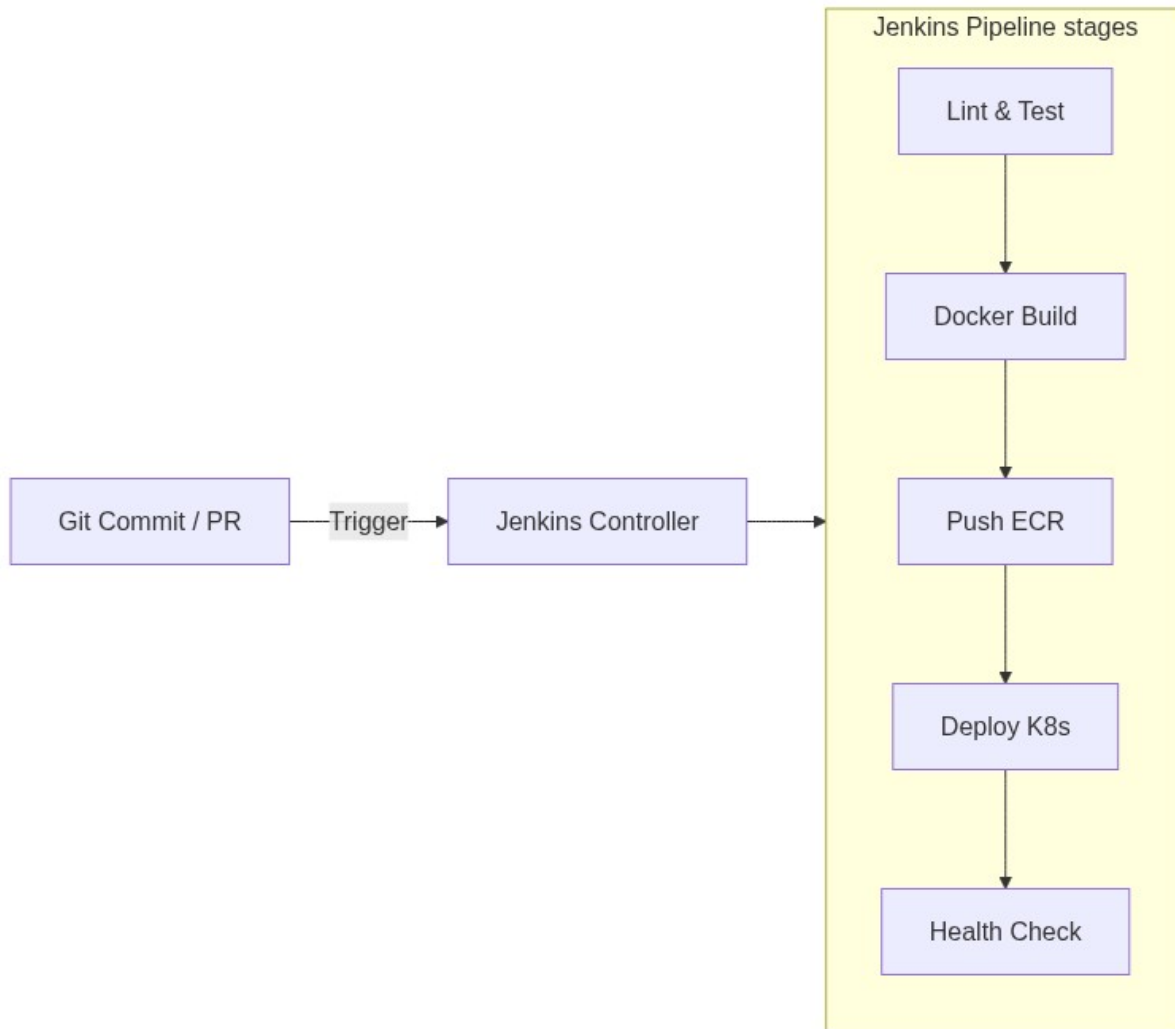
3. Infrastructure Architecture (AWS Production Target)

The production environment relies on Amazon Web Services (AWS) managed resources, set up via Terraform:



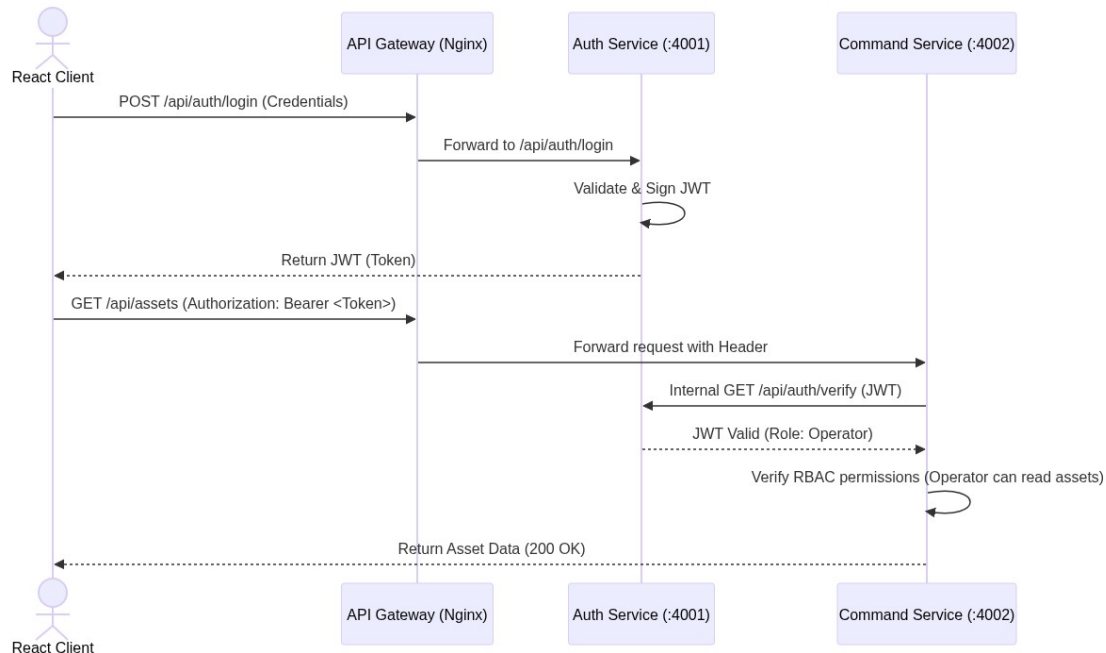
4. CI/CD Pipeline Flow (Jenkins)

The deployment pipeline automates validation, image compilation, registry pushing, and deployment rollout:



5. Identity Verification Flow

The sequence diagram below explains how user sessions are validated and authenticated dynamically across the microservice architecture:



5. Technical Details & Configuration

5.1. Authentication Service

- **Key Dependencies:** `express`, `jsonwebtoken`, `bcryptjs`, `@prisma/client`.
- **Password Hashing:** Passwords are cryptographically hashed using `bcryptjs` with a work factor of 12.
- **Token Structure:** JWTs are signed using the `HS256` algorithm. The token payload structure:

```

{
  "userId": 104,
  "name": "Jane Doe",
  "role": "Operator",
  "exp": 1781280000
}
  
```

- **Authentication Middleware:**

```

const jwt = require('jsonwebtoken');

const authenticateToken = (req, res, next) => {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) return res.status(401).json({ success: false, error: 'Unauthorized: Missing Token' });

  jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
    if (err) return res.status(403).json({ success: false, error: 'Forbidden: Invalid Token' });
    req.user = user;
    next();
  });
};
  
```

5.2. Command Service Telemetry Engine

Runs an interval loop every 3 seconds to update coordinate offsets and slowly drain fuel:

```
setInterval(async () => {
  const assets = await prisma.asset.findMany({ where: { status: 'ACTIVE' } });
  for (const asset of assets) {
    const deltaLat = (Math.random() - 0.5) * 0.005;
    const deltaLng = (Math.random() - 0.5) * 0.005;
    const newFuel = Math.max(0, asset.fuel - 0.1);

    const updated = await prisma.asset.update({
      where: { id: asset.id },
      data: {
        lat: asset.lat + deltaLat,
        lng: asset.lng + deltaLng,
        fuel: newFuel
      }
    });

    io.emit('telemetry:update', updated);
  }
}, 3000);
```

5.3. API Gateway Configuration (Nginx)

The gateway handles port redirection and WebSocket upgrades:

```
server {
  listen 80;
  server_name localhost;

  location / {
    root /usr/share/nginx/html;
    index index.html index.htm;
    try_files $uri $uri/ /index.html;
  }

  location /api/auth {
    proxy_pass http://auth-service:4001;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
  }

  location /api/domains {
    proxy_pass http://command-service:4002;
  }

  location /api/threats {
    proxy_pass http://threat-service:4003;
  }

  location /api/missions {
    proxy_pass http://mission-service:4004;
  }

  location /socket.io/ {
    proxy_pass http://command-service:4002;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "Upgrade";
    proxy_set_header Host $host;
  }
}
```

5.4. Docker Build Process (Multi-stage)

I use multi-stage Docker builds to keep image sizes small and remove development dependencies in production:

```
# Stage 1: Build & Install Dependencies
FROM node:22-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npx prisma generate
RUN npm prune --omit=dev

# Stage 2: Minimal Runtime Environment
FROM node:22-alpine AS runner
WORKDIR /app
COPY --from=builder /app/package*.json ./
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/src ./src
COPY --from=builder /app/prisma ./prisma

EXPOSE 4001
ENV NODE_ENV=production
USER node
CMD ["node", "src/index.js"]
```

5.5. Metrics & Observability Configuration

- **prom-client Integration:** Each Node.js microservice exposes Prometheus metrics on `/metrics`. A custom counter measures login volumes:

```
const client = require('prom-client');
const collectDefaultMetrics = client.collectDefaultMetrics;
collectDefaultMetrics({ register: client.register });

const loginCounter = new client.Counter({
  name: 'auth_service_login_total',
  help: 'Total login attempts on the Auth Service',
  labelNames: ['status']
});
```

- **Prometheus Target Configuration (prometheus.yml):**

```
scrape_configs:
- job_name: 'quantum-defense-services'
  scrape_interval: 10s
  static_configs:
    - targets: ['auth-service:4001', 'command-service:4002', 'threat-service:4003', 'mission-service:4004']
```

5.6. Vault Integration Architecture

Dynamic secret retrieval is managed in the microservice database configuration:

```
const vault = require('node-vault')({ endpoint: process.env.VAULT_ADDR });

async function getDatabaseCredentials() {
  const token = process.env.VAULT_TOKEN;
  vault.token = token;

  const secrets = await vault.read('secret/data/quantum-defense/postgres');
  const { username, password, host, database } = secrets.data.data;

  return `postgresql://${username}:${password}@${host}:5432/${database}?schema=public`;
}
```

5.7. Environment Variables Reference

Below is the configuration checklist required in `.env` configuration files:

---	---	---	---
PORT	Microservice binding port	4001-4004	All Services
DATABASE_URL	PostgreSQL direct connection URI	postgresql://postgres:postgres@localhost:5432/qdefense	All Services
JWT_SECRET	Token signature seed	c2-top-secret-signing-key	Auth Service
VAULT_ADDR	Connection address of Secrets Server	http://vault:8200	All Services
VAULT_TOKEN	Local authentication token	root-dev-token	All Services
SOCKET_IO_PORT	WebSockets binding	4002	Command, Threat

■ End-to-End Operational Manual (Command Reference)

I have documented each command and script execution below to guide developers and system operators through the deployment and teardown lifecycles.

1. Local Development Operations

Ensure Docker Desktop is active on your host system:

```
# 1. Clone the repository and navigate to the directory
git clone https://github.com/HakashiKatake/QuantumDefence.git
cd QuantumDefence

# 2. Copy and configure the local environment file
cp scripts/secrets.env.example scripts/secrets.env

# 3. Spin up PostgreSQL and Vault containers
./scripts/db-setup.sh

# 4. Initialize Vault engines, policies, and secrets
./vault/scripts/init-vault.sh

# 5. Compile and start all 13 containers in the dev network
docker compose -f docker/docker-compose.yml build
docker compose -f docker/docker-compose.yml up -d

# 6. Verify container runtime status
docker compose -f docker/docker-compose.yml ps

# 7. Start React client development server
```

```
cd frontend
npm install
npm run dev
```

- **Application Interface:** Open <http://localhost:3000> to access the Vite development hot-reload server.
- **Database Management:** View metrics at <http://localhost:9090> (Prometheus) and logs at <http://localhost:5601> (Kibana).

2. Production AWS Cloud Operations

Authenticate with your AWS CLI and run the following deployment commands:

```
# 1. Set up AWS configuration
aws configure
aws sts get-caller-identity

# 2. Provision AWS resources via Terraform
cd terraform
terraform init
terraform workspace select production || terraform workspace new production
terraform plan -out=tfplan
terraform apply tfplan

# 3. Pull kubeconfig permissions for kubectl EKS management
aws eks update-kubeconfig --name quantum-defense-cluster --region us-east-1
kubectl get nodes

# 4. Install Vault in EKS namespace using Helm
helm repo add hashicorp https://helm.releases.hashicorp.com
helm repo update
helm upgrade --install vault hashicorp/vault --namespace quantum-defense --set "server.dev.enabled=true" --set "server.enabled=true"
kubectl wait --for=condition=Ready pod/vault-0 -n quantum-defense --timeout=120s

# 5. Populate Vault database and JWT secrets on EKS
kubectl exec -it vault-0 -n quantum-defense -- vault kv put secret/quantum-defense/postgres \
  username="postgres" \
  password="TacticalC2SecureDBPass!" \
  host="rds-postgres-endpoint.amazonaws.com" \
  database="qdefense"

# 6. Install Nginx Ingress Controller
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update
helm upgrade --install ingress-nginx ingress-nginx/ingress-nginx --namespace ingress-nginx --create-namespace
kubectl get service ingress-nginx-controller -n ingress-nginx

# 7. Deploy Prometheus and Grafana
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm upgrade --install monitoring prometheus-community/kube-prometheus-stack --namespace monitoring --create-namespace
kubectl apply -f ../monitoring/monitoring-ingress.yaml

# 8. Authenticate and push compiled Docker images to ECR
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 032667094119.dkr.ecr.us-east-1.amazonaws.com

SERVICES=("auth-service" "command-service" "threat-service" "mission-service" "frontend")
REGISTRY="032667094119.dkr.ecr.us-east-1.amazonaws.com/quantum-defense"
for SVC in "${SERVICES[@]}; do
  if [ "$SVC" == "frontend" ]; then
    docker build -t "$REGISTRY/$SVC:latest" ./frontend
  else
    docker build -t "$REGISTRY/$SVC:latest" ./services/$SVC
  fi
  docker push "$REGISTRY/$SVC:latest"
```

done

```
# 9. Apply Kubernetes Deployments, Services, and HPAs
cd ../kubernetes
kubectl apply -f namespace.yaml
kubectl apply -f app/configmap.yaml
kubectl apply -f app/auth-deployment.yaml
kubectl apply -f app/command-deployment.yaml
kubectl apply -f app/threat-deployment.yaml
kubectl apply -f app/mission-deployment.yaml
kubectl apply -f app/frontend-deployment.yaml
kubectl apply -f app/ingress.yaml
kubectl apply -f app/hpa.yaml

# 10. Run database migrations and seeding against AWS RDS database
kubectl port-forward svc/auth-service 4001:4001 -n quantum-defense &
PID=$!
cd ../services/auth-service
DATABASE_URL="postgresql://postgres:TacticalC2SecureDBPass!@localhost:5432/qdefense?schema=auth" npx prisma db push
DATABASE_URL="postgresql://postgres:TacticalC2SecureDBPass!@localhost:5432/qdefense?schema=auth" npx prisma db seed
kill $PID
```

■■ Disaster Recovery (DR) Plan

I established a complete Disaster Recovery plan to ensure military operational continuity:

- **Recovery Time Objective (RTO): 15 Minutes** (maximum duration to recover from a primary region failure).
- **Recovery Point Objective (RPO): 5 Minutes** (maximum acceptable data loss window).

Playbook 1: Single Availability Zone Outage (Auto-Failover)

- **RDS Failover:** Amazon RDS automatically performs DNS failover to the standby database instance in the secondary AZ within 1-2 minutes. No manual intervention is needed.
- **EKS Self-Healing:** Managed node groups automatically spin up fresh pods in the healthy AZ.

Playbook 2: Primary Region Outage (Cross-Region Failover)

- **IaC Re-provisioning:**

```
cd terraform
terraform workspace select backup-region || terraform workspace new backup-region
terraform apply -auto-approve
```

- **Kubeconfig Re-routing:**

```
aws eks update-kubeconfig --name quantum-defense-cluster --region us-west-2
kubectl apply -f kubernetes/namespace.yaml
kubectl apply -f kubernetes/app/
```

- **Traffic Shift:** Route 53 DNS records are updated to shift active global traffic to the backup region's load balancer.

Playbook 3: Complete Teardown & Zero-Cost Guard

To safely clean up all AWS resources and return my active billing footprint to exactly **\$0.00**, I run the automated script:

```
./scripts/prod-cleanup.sh
```


This executes the following steps:

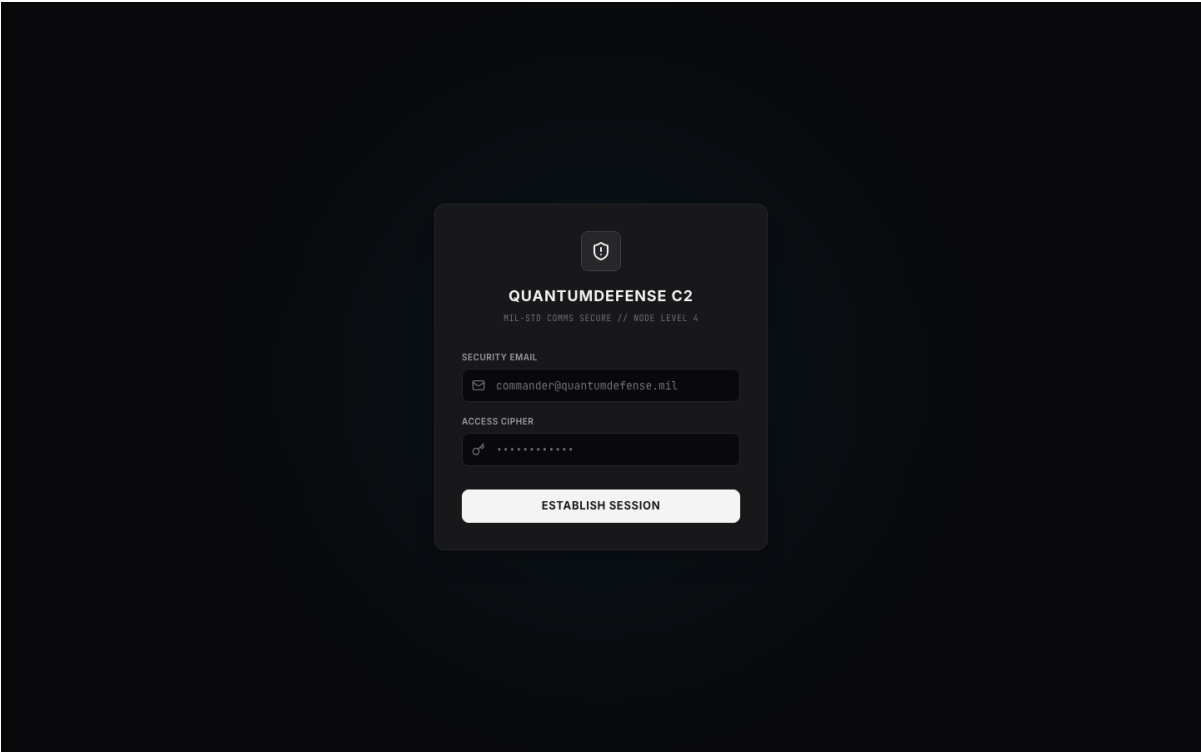
1. Deletes the EKS namespace to release associated LoadBalancers.
2. Uninstalls Prometheus, Grafana, and Vault Helm charts.
3. Recursively deletes objects inside the S3 backup bucket.
4. Purges all ECR repository images.
5. Executes `terraform destroy -auto-approve` to destroy EKS, RDS, EC2, and VPC instances.

■ ■ Demonstration Screenshots

Demonstration screenshots are located in `docs/screenshots/`.

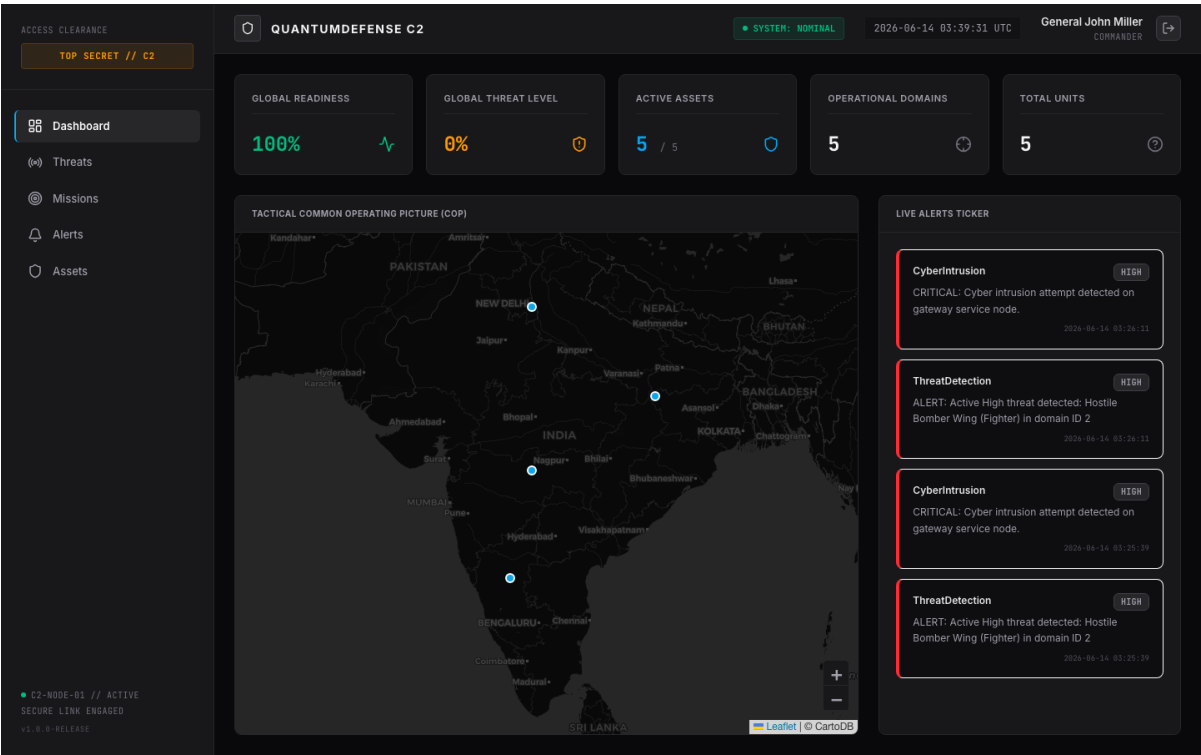
1. [Secure Login Interface](#): Authenticates command personnel using credentials stored in PostgreSQL and validated via JWT.
2. [Tactical COP Dashboard](#): Visualizes domain assets, threat alerts, and coordinates on the Leaflet.js map.
3. [Active Threat Tracker](#): Shows severity classifications and neutralization statuses.
4. [Mission Scheduling](#): Directs state transitions and assigns unit targets.
5. [Asset Telemetry & Readiness](#): Tracks readiness, coordinates, and fuel levels of deployment assets.
6. [Alert & Notification Stream](#): Broadcasts real-time security threats and system log events via Socket.IO.
7. [Vault Administration](#): Demonstrates secret storage and credentials protection.
8. [Jenkins CI/CD Pipeline Running](#): Illustrates the Jenkins pipeline execution in progress.
9. [Jenkins CI/CD Pipeline Success](#): Illustrates the successful build and deploy pipeline stages.
10. [Grafana Metrics Monitoring](#): Details container memory, CPU utilization, and HTTP request throughput.
11. [Kibana Logging Query](#): Queries structured JSON logs from microservices.
12. [AWS Production Architecture](#): Visualizes the cloud infrastructure, VPC layout, EKS cluster, ECR registries, RDS PostgreSQL database, S3 backups, and CloudWatch integration.

1. Secure Login Interface



Secure Login Interface

2. Tactical COP Dashboard



Tactical COP Dashboard

3. Active Threat Tracker

ACCESS CLEARANCE

TOP SECRET // C2

Dashboard

Threats

Missions

Alerts

Assets

C2-NODE-01 // ACTIVE

SECURE LINK ENGAGED

V1.0.0-RELEASE

QUANTUMDEFENSE C2

SYSTEM: NOMINAL

2026-06-14 03:39:34 UTC

General John Miller
COMMANDER

Multi-Domain Threat Management

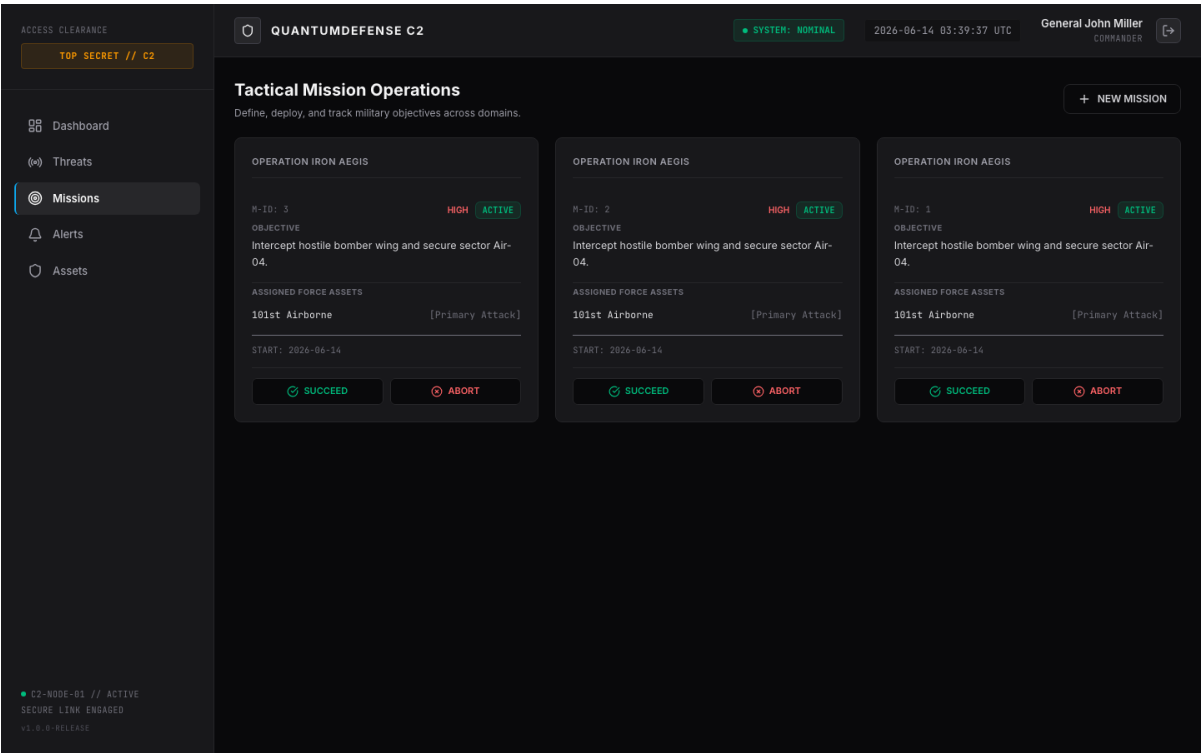
Monitor, register, and coordinate neutralization of hostile targets.

ACTIVE HOSTILE TARGET REGISTER

TARGET ID	DESIGNATION	SIGNATURE TYPE	SEVERITY	COORDINATES	STATUS	DETECTED TIME	ACTIONS
T-2	Hostile Bomber Wing	Fighter	HIGH	22.5000, 76.8000	DETECTED	6/14/2026, 8:56:11 AM	NEUTRALIZE
T-1	Hostile Bomber Wing	Fighter	HIGH	22.5000, 76.8000	DETECTED	6/14/2026, 8:55:38 AM	NEUTRALIZE

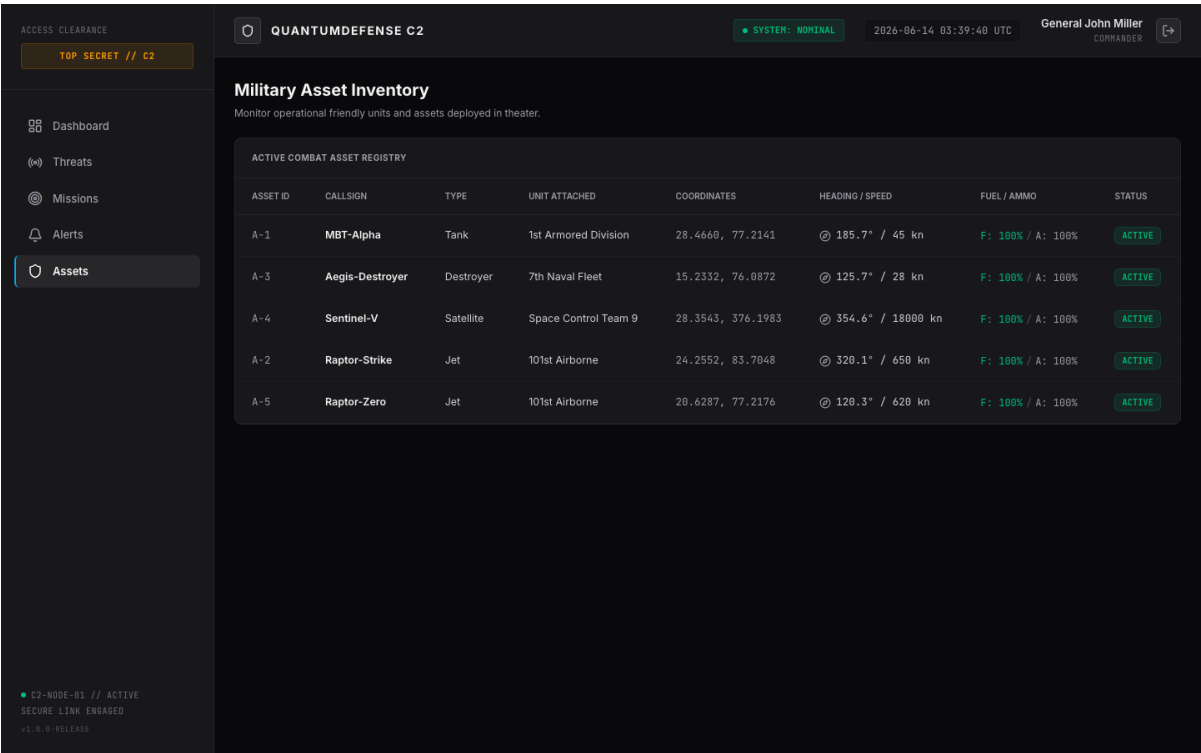
Active Threat Tracker

4. Mission Scheduling



Mission Scheduling

5. Asset Telemetry & Readiness



Asset Telemetry & Readiness

6. Alert & Notification Stream

ACCESS CLEARANCE

TOP SECRET // C2

Dashboard

Threats

Missions

Alerts

Assets

C2-NODE-01 // ACTIVE

SECURE LINK ENGAGED

V1.0.0-RELEASE

QUANTUMDEFENSE C2

SYSTEM: NOMINAL

2026-06-14 03:39:43 UTC

General John Miller
COMMANDER

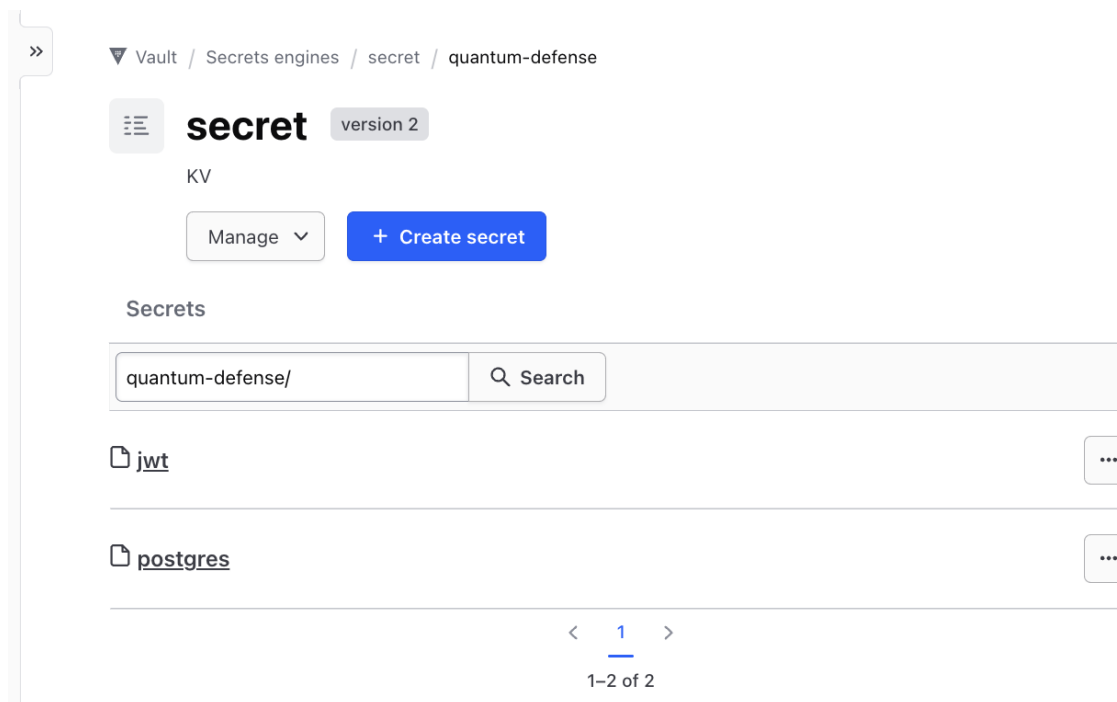
Real-Time Security Alerts

Acknowledge warning indicators and security notifications.

ALERT ID	INCIDENT TYPE	SEVERITY	DETAILS MESSAGE	TRIGGER TIME	STATUS	ACTIONS
A-4	CyberIntrusion	HIGH	CRITICAL: Cyber intrusion attempt detected on gateway service node.	6/14/2026, 8:56:11 AM	UNACKNOWLEDGED	ACKNOWLEDGE
A-3	ThreatDetection	HIGH	ALERT: Active High threat detected: Hostile Bomber Wing (Fighter) in domain ID 2	6/14/2026, 8:56:11 AM	UNACKNOWLEDGED	ACKNOWLEDGE
A-2	CyberIntrusion	HIGH	CRITICAL: Cyber intrusion attempt detected on gateway service node.	6/14/2026, 8:56:39 AM	UNACKNOWLEDGED	ACKNOWLEDGE
A-1	ThreatDetection	HIGH	ALERT: Active High threat detected: Hostile Bomber Wing (Fighter) in domain ID 2	6/14/2026, 8:56:39 AM	UNACKNOWLEDGED	ACKNOWLEDGE

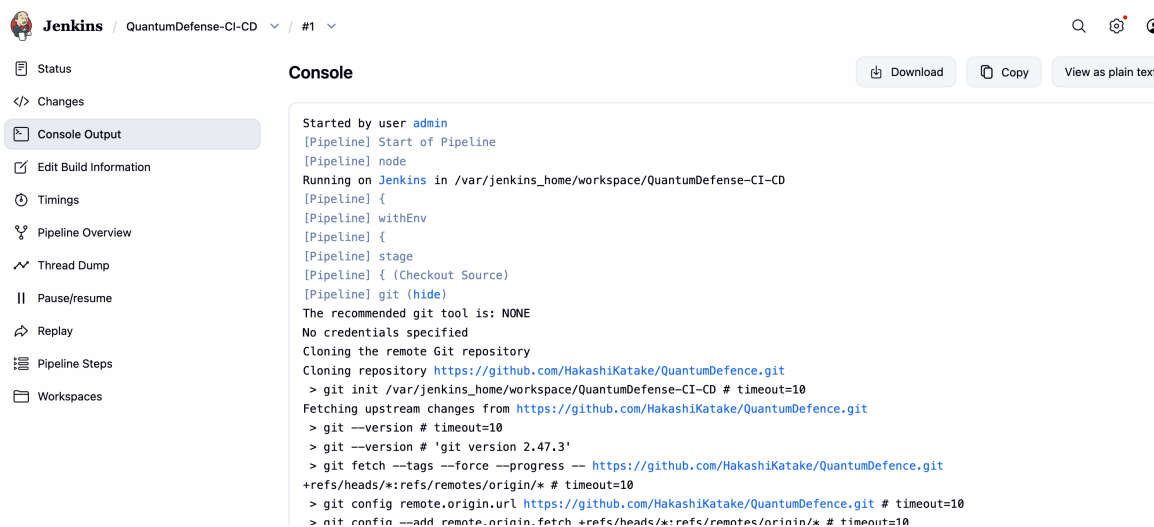
Alert & Notification Stream

7. Vault Administration



Vault Administration

8. Jenkins CI/CD Pipeline Running



Jenkins CI/CD Pipeline Running

9. Jenkins CI/CD Pipeline Success



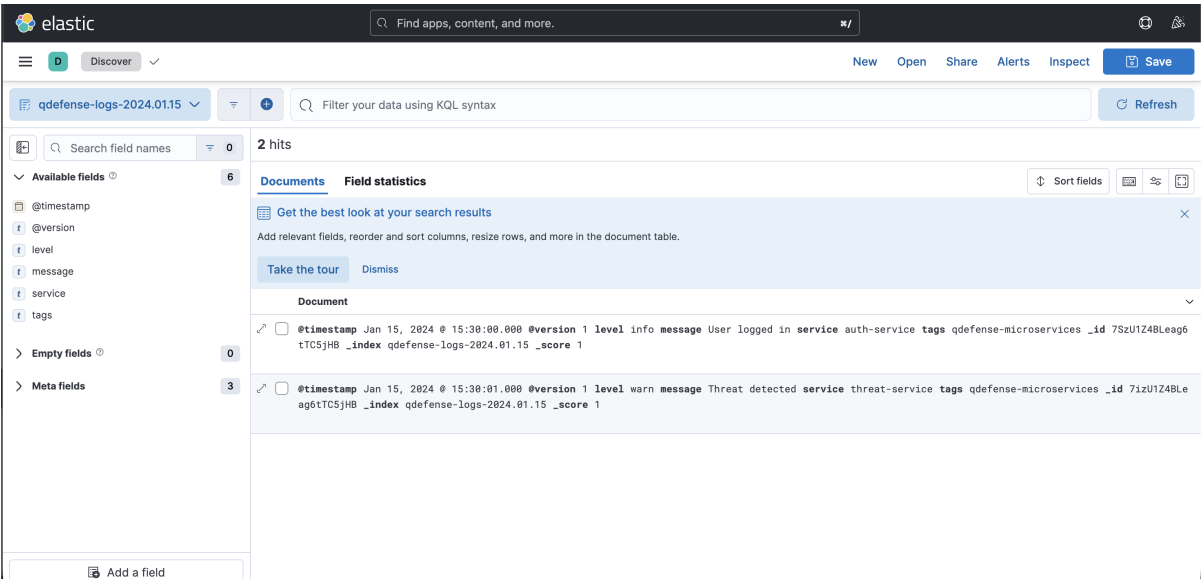
Jenkins CI/CD Pipeline Success

10. Grafana Metrics Monitoring



Grafana Metrics Monitoring

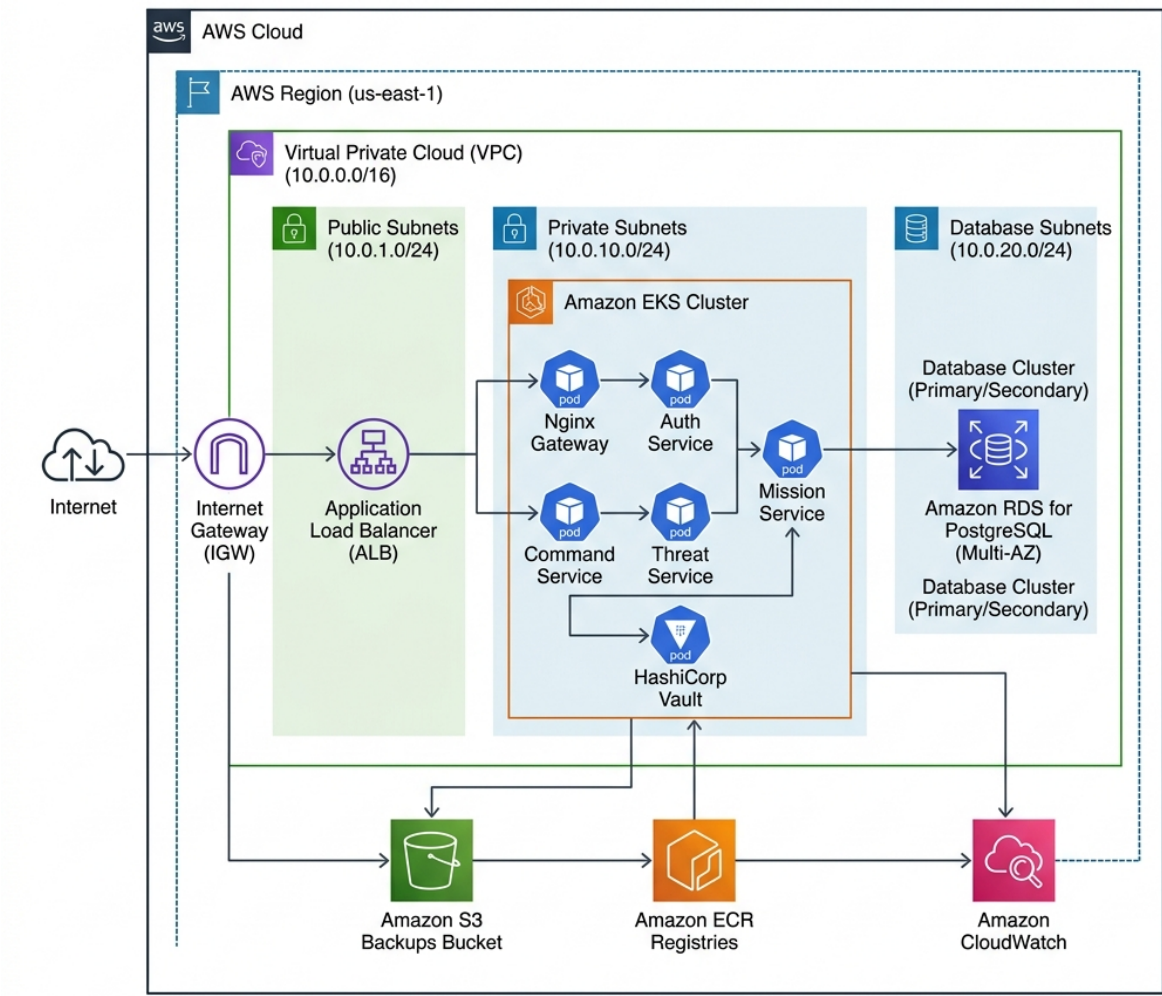
11. Kibana Logging Query



Kibana Logging Query

12. AWS Production Architecture

AWS Architecture Diagram: QuantumDefense Platform



AWS Production Architecture

■ Complete Scripts Reference & Workflow Documentation

Local Development Scripts

---	---	---
-----	-----	-----

<code>start-dev.sh</code>	<code>scripts/start-dev.sh</code>	Starts all Docker containers (13 services) with <code>--build</code>
<code>stop-dev.sh</code>	<code>scripts/stop-dev.sh</code>	Gracefully stops containers without removing volumes
<code>clean-dev.sh</code>	<code>scripts/clean-dev.sh</code>	Full cleanup: stop containers, remove volumes, prune images
<code>db-setup.sh</code>	<code>scripts/db-setup.sh</code>	Launches PostgreSQL/Vault, pushes Prisma schemas, seeds initial data
<code>db-seed.sh</code>	<code>scripts/db-seed.sh</code>	Production-only: pushes schemas/seeds to RDS via temporary pods
<code>init-vault.sh</code>	<code>vault/scripts/init-vault.sh</code>	Configures KV v2 engine, writes postgres/jwt secrets, applies policies
<code>install-deps.sh</code>	<code>scripts/install-deps.sh</code>	Local npm install + Prisma client generation for all services

Production Scripts

---	---	---
<code>prod-provision.sh</code>	<code>scripts/prod-provision.sh</code>	Full AWS deployment: VPC, EKS, RDS, Jenkins, monitoring, apps
<code>prod-cleanup.sh</code>	<code>scripts/prod-cleanup.sh</code>	Complete AWS teardown: namespaces, helm, ECR, terraform destroy

Local Development Workflow (Step-by-Step)

```
# 1. Prerequisites Check
# - Docker Desktop running
# - Node.js 20+ installed
# - AWS CLI configured (for ECR login during build)

# 2. Install Service Dependencies
./scripts/install-deps.sh
# - Runs npm install in each service directory
# - Generates Prisma clients

# 3. Setup Database & Vault
./scripts/db-setup.sh
# - docker compose up -d postgres vault
# - Waits for PostgreSQL health
# - Pushes Prisma schemas (auth, command, threat, mission)
# - Seeds initial military personnel
# - Inserts default domains/units/assets

# 4. Initialize Vault Secrets
./vault/scripts/init-vault.sh
# - Enables KV v2 secrets engine at path 'secret'
# - Writes: secret/quantum-defense/postgres (credentials)
# - Writes: secret/quantum-defense/jwt (signing key)
# - Applies 'c2-policy' (read-only for quantum-defense/*)

# 5. Build All Container Images
docker compose -f docker/docker-compose.yml build

# 6. Start Full Development Environment
docker compose -f docker/docker-compose.yml up -d
# Or use the script:
./scripts/start-dev.sh

# 7. Verify Running Services
docker compose -f docker/docker-compose.yml ps
# Expected: 13 containers running

# 8. Access Points
# Frontend: http://localhost:3000 (Vite dev) or http://localhost:80 (built)
# Auth API: http://localhost:4001/api/auth
# Command API: http://localhost:4002/api/domains
# Threat API: http://localhost:4003/api/threats
# Mission API: http://localhost:4004/api/missions
# Vault: http://localhost:8200 (token: root-dev-token)
# Prometheus: http://localhost:9090
# Grafana: http://localhost:3001
# Kibana: http://localhost:5601
# Elasticsearch: http://localhost:9200

# 9. Stop Development Environment (Partial)
./scripts/stop-dev.sh # Keeps volumes and images

# 10. Full Cleanup (When done)
./scripts/clean-dev.sh # Removes everything
```

Production Workflow (Step-by-Step)

```
# 1. Configure AWS Credentials
aws configure
aws sts get-caller-identity

# 2. Setup Environment Secrets
cp scripts/secrets.env.example scripts/secrets.env
# Edit scripts/secrets.env with real values:
# - VAULT_DEV_TOKEN
# - DB_PASSWORD
# - JWT_SECRET
# - GRAFANA_PASSWORD
# - JENKINS_AWS_KEY_ID/SECRET

# 3. Deploy Infrastructure
cd terraform
terraform init
terraform workspace select production || terraform workspace new production
terraform apply -auto-approve

# 4. Configure kubectl
aws eks update-kubeconfig --name quantum-defense-cluster --region us-east-1

# 5. Deploy Vault via Helm
helm repo add hashicorp https://helm.releases.hashicorp.com
helm repo update
helm upgrade --install vault hashicorp/vault \
  --set "server.dev.enabled=true" \
  --set "server.dev.devRootToken=${VAULT_DEV_TOKEN}" \
  -n quantum-defense --create-namespace

# 6. Wait for Vault
kubectl wait --for=condition=Ready pod/vault-0 -n quantum-defense --timeout=180s

# 7. Deploy Nginx Ingress
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm install ingress-nginx ingress-nginx/ingress-nginx -n ingress-nginx --create-namespace

# 8. Build & Push Images to ECR
./scripts/prod-provision.sh # Step 5: auto-builds and pushes to ECR

# 9. Apply Kubernetes Manifests
kubectl apply -f kubernetes/namespace.yaml
kubectl apply -f kubernetes/app/configmap.yaml
kubectl apply -f kubernetes/app/ingress.yaml
kubectl apply -f kubernetes/app/hpa.yaml
kubectl apply -f kubernetes/app/auth-deployment.yaml
kubectl apply -f kubernetes/app/command-deployment.yaml
kubectl apply -f kubernetes/app/threat-deployment.yaml
kubectl apply -f kubernetes/app/mission-deployment.yaml
kubectl apply -f kubernetes/app/frontend-deployment.yaml

# 10. Seed Database
./scripts/db-seed.sh

# 11. Deploy Monitoring Stack
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring --create-namespace \
  --values monitoring/helm-values.yaml \
  --set grafana.adminPassword="${GRAFANA_PASSWORD}"

# 12. Teardown (Zero-Cost Guard)
./scripts/prod-cleanup.sh
```

Service Port Reference

---	---	---	---
auth-service	4001	4001	auth
command-service	4002	4002	command
threat-service	4003	4003	threat
mission-service	4004	4004	mission
postgres	5432	5432	qdefense (multi-schema)
vault	8200	8200	KV v2 engine
prometheus	9090	9090	-
grafana	3001	3000	-
kibana	5601	5601	-
elasticsearch	9200	9200	-
logstash	50000	50000	-
gateway/nginx	80	80	-

Docker Compose Stack Services

```
# docker/docker-compose.yml services:
- postgres (postgresql:18-alpine)
- vault (hashicorp/vault:latest)
- auth-service (build: services/auth-service)
- command-service (build: services/command-service)
- threat-service (build: services/threat-service)
- mission-service (build: services/mission-service)
- gateway (nginx:stable-alpine)
- prometheus (prom/prometheus:latest)
- grafana (grafana/grafana:latest)
- elasticsearch (docker.elastic.co/elasticsearch/elasticsearch:8.12.0)
- logstash (docker.elastic.co/logstash/logstash:8.12.0)
- kibana (docker.elastic.co/kibana/kibana:8.12.0)
```

Vault Secrets Structure

```
secret/
  -- quantum-defense/
    -- postgres
    |   -- username: "postgres"
    |   -- password: "postgrespassword"
    |   -- host: "postgres"
    |   -- database: "qdefense"
  -- jwt
  -- secret: "c2-top-secret-signing-key"
```

Default Credentials (Development Only)

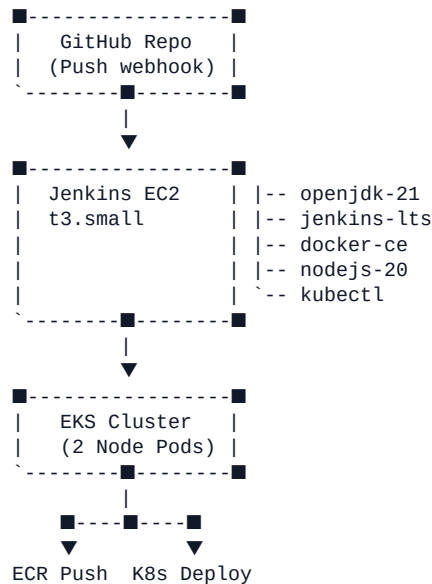
- Vault Token: `root-dev-token`
- JWT Secret: `c2-top-secret-signing-key`
- PostgreSQL: `postgres` / `postgrespassword`
- Jenkins Admin: Check `initialAdminPassword` via SSM after first boot

Troubleshooting

---	---
Kibana shows "no logs found"	Refresh index pattern in Stack Management, check <code>qdefense-logs-*</code> exists
Vault secrets empty	Run <code>./vault/scripts/init-vault.sh</code> after containers start
Database connection refused	Wait for PostgreSQL health check, verify <code>DATABASE_URL</code> env vars
Docker build fails	Run <code>npm install</code> in service directory, check lockfile sync
Vault connection in containers	Verify <code>VAULT_ADDR=http://vault:8200</code> (container network)

Jenkins CI/CD Pipeline Documentation

Jenkins Architecture



Jenkins Pipeline Stages (Jenkinsfile)

Located in `jenkins/Jenkinsfile`:

---	---
Checkout Source	Clones <code>main</code> branch from GitHub
Static Analysis & Test	Parallel ESLint on all 5 services (auth, command, threat, mission, frontend)
Docker Compilation	Builds all 5 container images locally
Registry Upload	ECR authentication + push with AWS credentials binding
Kubernetes Orchestration	Applies namespace, configmap, 5 deployments, ingress, HPA; restarts rollouts
Health Validation	Waits for all deployments to be ready (90s timeout each)

Jenkins Setup (Automated)

`prod-provision.sh` Step 10 performs:

1. Fetches initial admin password via SSM
2. Installs plugins: `aws-credentials`, `kubernetes-cli`, `pipeline-stage-view`, `git`, `workflow-aggregator`

3. Creates AWS credentials (aws-ecr-credentials)
4. Creates kubeconfig secret (eks-kubeconfig)
5. Creates pipeline job from jenkins/Jenkinsfile

Manual Jenkins Access

```
# Get Jenkins public IP
JENKINS_IP=$(terraform -chdir=terraform output -raw jenkins_public_ip)
echo "Jenkins URL: http://${JENKINS_IP}:8080"

# Get initial admin password (SSM command required)
aws ssm send-command \
  --instance-ids $(aws ec2 describe-instances --filters "Name=tag:Name,Values=quantum-defense-jenkins" --query 'Res
  --document-name "AWS-RunShellScript" \
  --parameters 'commands=["cat /var/lib/jenkins/secrets/initialAdminPassword"]'

# Configure webhook (add to GitHub repo settings)
# http://${JENKINS_IP}:8080/github-webhook/
```

Jenkins Credentials Required

---	---	---
aws-ecr-credentials	AWS Credentials	ECR push access
eks-kubeconfig	Secret File	kubectl cluster access

Secrets Configuration

Copy scripts/secrets.env.example to scripts/secrets.env and fill in:

---	---	---
DB_PASSWORD	RDS, Vault, Services	PostgreSQL database password
JWT_SECRET	Vault, Auth Service	JWT token signing key

API Endpoints Reference

---	---	---	---
Auth	/api/auth/register	POST	Register new user with name, email, password, role
Auth	/api/auth/login	POST	Authenticate user, returns JWT token
Auth	/api/auth/verify	GET	Internal JWT validation (used by other services)

Auth	/api/auth/health	GET	Health check endpoint
Command	/api/domains	GET	List all military domains
Command	/api/assets	GET	List all assets with telemetry
Command	/api/units	GET	List all military units
Command	/api/dashboard	GET	Combined dashboard statistics
Threat	/api/threats	GET	List detected threats
Threat	/api/alerts	GET	List security alerts
Mission	/api/missions	GET	List mission directives
Mission	/api/missions/:id	PUT	Update mission status

WebSocket Endpoints

---	---	---	---
Command	telemetry:update	Server → Client	Asset position/fuel updates every 3s
Threat	threat:detected	Server → Client	Real-time threat alerts

Kubernetes Resources

---	---	---
namespace.yaml	kubernetes/	Creates quantum-defense namespace
hpa.yaml	kubernetes/app/	Autoscaling for command-service (2-10 pods), threat-service (2-8 pods)
ingress.yaml	kubernetes/app/	Nginx Ingress routing rules
configmap.yaml	kubernetes/app/	Environment variables for services
auth-deployment.yaml	kubernetes/app/	Auth service Deployment + Service (2 replicas)
command-deployment.yaml	kubernetes/app/	Command service Deployment + Service (2 replicas)

threat-deployment.yaml	kubernetes/app/	Threat service Deployment + Service (2 replicas)
mission-deployment.yaml	kubernetes/app/	Mission service Deployment + Service (2 replicas)
frontend-deployment.yaml	kubernetes/app/	React frontend Deployment (2 replicas)
monitoring-ingress.yaml	monitoring/	Grafana/Prometheus route paths

■ Future Improvements

While Project QuantumDefense represents a fully functional C2 platform, the following engineering enhancements are planned:

1. **Vault Dynamic Database Credentials:** Configure Vault to generate dynamic, short-lived PostgreSQL credentials that automatically expire and rotate every 15 minutes, rather than relying on static passwords.
2. **Distributed Tracing (OpenTelemetry):** Integrate OpenTelemetry SDKs into the microservices to trace requests across the API Gateway, Auth Service, and Command Service, helping locate performance bottlenecks.
3. **Active-Active Multi-Region Database:** Transition from active-passive cross-region backup to a true active-active multi-region deployment using Amazon Aurora Global Database to achieve RTO/RPO near zero.
4. **Distroless Container Hardening:** Swap the microservices' base Alpine Docker images for Google's distroless images to remove package managers, shells, and minimize the container attack surface.
5. **Helm Chart Integration:** Consolidate local Kubernetes manifests into custom Helm charts to make deployment configuration parameters easily customizable across environments.